Reflection Report: Personalized GitHub Repository Agent

I certify that this reflection was written entirely by me without the use of generative AI tools.

This report explains how a GitHub Repository Agent was built and the overall analysis of the effectiveness of each module. The Agent was built in a way that it would review the code changes on a Git repository, draft the issues and pull requests with human approval, and finally solve existing issues. The framework was built with the help of four agentic patterns, such as, Planning, Tool Use, Reflection and Multi-Agent design. The multiple agents involved were Reviewer, Planner, Writer and Gatekeeper. This separated architecture makes reasoning better, with a higher accuracy or efficiency of outputs and removing hallucination.

The role of each of these agents were pre-decided. For example, reviewer uses git tools like git diff and identifies any potential issues. The planner would then act upon these issues and devise a plan of action. Nothing gets drafted by the Writer until a plan exists. The LLM is forced to answer specific questions related to planning with a fixed prompt as seen in the code. It ensures recommendation, risk level and provide clear hints for the Writer Agent. Reflection pattern is seen through gatekeeper as it runs a critique step that produces Reflection Result. The Gatekeepeer Agent has separate reflection methods for separate tasks like reflection of an issue, reflection of PL, etc. This stage leads the LLM to not generate outputs without proper reasoning. The orchestrator runs them in a sequence so that no Agent is skipped. While there are many advantages of this separated architecture, but the trade-offs include higher context limit as the output of each of the agent is passed down to the next.

The Planning pattern removed hallucination to a great extent as a plan of action was first drafted. Otherwise, issues like hallucinating non-existent files or generating outputs out of the required constraints were very common. Grounding through tool use led the Agent to pinpoint issues rather than hallucinating them. Since the Agent was using Python tool calls, it was making its decision based on real evidence. I don't feel there is much risk left after tool grounding, apart from the variability of how an LLM would respond to the structured results. Reflection pattern was implemented through Gatekeeper, and it definitely improved the proposal quality. This is because the same model that was used to write the draft was also responsible for evaluating it. For example, a draft asked to implement rate limiting for the login endpoint. The Gatekeeper flagged it since the only evidence was .DS_Store file was staged for commit and no changes were found during code review. It then asked for further explanation of acceptance criteria, references such as repository lines, commit links or other supporting data.

One of the limitations would be not storing any memory of the issues. Each time the agent is asked to run the GitHub repository from the very beginning and implement each stage. With a memory or RAG alike architecture, the Agent can learn from previous issues and become more robust in decision making.